

```
>>> S + str(I)           # Force concatenation
'421'
```

Similar built-in functions handle floating-point number conversions to and from strings:

```
>>> str(3.1415), float("1.5")
('3.1415', 1.5)

>>> text = "1.234E-10"
>>> float(text)
1.2340000000000001e-010
```

Later, we'll further study the built-in `eval` function; it runs a string containing Python expression code and so can convert a string to any kind of object. The functions `int` and `float` convert only to numbers, but this restriction means they are usually faster (and more secure, because they do not accept arbitrary expression code). As we saw briefly in [Chapter 5](#), the string formatting expression also provides a way to convert numbers to strings. We'll discuss formatting further later in this chapter.

Character code conversions

On the subject of conversions, it is also possible to convert a single character to its underlying ASCII integer code by passing it to the built-in `ord` function—this returns the actual binary value of the corresponding byte in memory. The `chr` function performs the inverse operation, taking an ASCII integer code and converting it to the corresponding character:

```
>>> ord('s')
115
>>> chr(115)
's'
```

You can use a loop to apply these functions to all characters in a string. These tools can also be used to perform a sort of string-based math. To advance to the next character, for example, convert and do the math in integer:

```
>>> S = '5'
>>> S = chr(ord(S) + 1)
>>> S
'6'
>>> S = chr(ord(S) + 1)
>>> S
'7'
```

At least for single-character strings, this provides an alternative to using the built-in `int` function to convert from string to integer:

```
>>> int('5')
5
>>> ord('5') - ord('0')
5
```

Such conversions can be used in conjunction with looping statements, introduced in [Chapter 4](#) and covered in depth in the next part of this book, to convert a string of binary digits to their corresponding integer values. Each time through the loop, multiply the current value by 2 and add the next digit's integer value:

```
>>> B = '1101'                # Convert binary digits to integer with ord
>>> I = 0
>>> while B != '':
...     I = I * 2 + (ord(B[0]) - ord('0'))
...     B = B[1:]
...
>>> I
13
```

A left-shift operation (`I << 1`) would have the same effect as multiplying by 2 here. We'll leave this change as a suggested exercise, though, both because we haven't studied loops in detail yet and because the `int` and `bin` built-ins we met in [Chapter 5](#) handle binary conversion tasks for us in Python 2.6 and 3.0:

```
>>> int('1101', 2)            # Convert binary to integer: built-in
13
>>> bin(13)                   # Convert integer to binary
'0b1101'
```

Given enough time, Python tends to automate most common tasks!

Changing Strings

Remember the term “immutable sequence”? The *immutable* part means that you can't change a string in-place (e.g., by assigning to an index):

```
>>> S = 'spam'
>>> S[0] = "x"
Raises an error!
```

So, how do you modify text information in Python? To change a string, you need to build and assign a new string using tools such as concatenation and slicing, and then, if desired, assign the result back to the string's original name:

```
>>> S = S + 'SPAM!'           # To change a string, make a new one
>>> S
'spamSPAM!'
>>> S = S[:4] + 'Burger' + S[-1]
>>> S
'spamBurger!'
```

The first example adds a substring at the end of `S`, by concatenation (really, it makes a new string and assigns it back to `S`, but you can think of this as “changing” the original string). The second example replaces four characters with six by slicing, indexing, and concatenating. As you'll see in the next section, you can achieve similar effects with string method calls like `replace`:

```
>>> S = 'sploit'
>>> S = S.replace('pl', 'pamal')
>>> S
'spamalot'
```

Like every operation that yields a new string value, string methods generate new string objects. If you want to retain those objects, you can assign them to variable names. Generating a new string object for each string change is not as inefficient as it may sound—remember, as discussed in the preceding chapter, Python automatically garbage collects (reclaims the space of) old unused string objects as you go, so newer objects reuse the space held by prior values. Python is usually more efficient than you might expect.

Finally, it's also possible to build up new text values with string formatting expressions. Both of the following substitute objects into a string, in a sense converting the objects to strings and changing the original string according to a format specification:

```
>>> 'That is %d %s bird!' % (1, 'dead')           # Format expression
That is 1 dead bird!
>>> 'That is {0} {1} bird!'.format(1, 'dead')     # Format method in 2.6 and 3.0
'That is 1 dead bird!'
```

Despite the substitution metaphor, though, the result of formatting is a new string object, not a modified one. We'll study formatting later in this chapter; as we'll find, formatting turns out to be more general and useful than this example implies. Because the second of the preceding calls is provided as a method, though, let's get a handle on string method calls before we explore formatting further.



As we'll see in [Chapter 36](#), Python 3.0 and 2.6 introduce a new string type known as `bytearray`, which is mutable and so may be changed in place. `bytearray` objects aren't really strings; they're sequences of small, 8-bit integers. However, they support most of the same operations as normal strings and print as ASCII characters when displayed. As such, they provide another option for large amounts of text that must be changed frequently. In [Chapter 36](#) we'll also see that `ord` and `chr` handle Unicode characters, too, which might not be stored in single bytes.

String Methods

In addition to expression operators, strings provide a set of *methods* that implement more sophisticated text-processing tasks. Methods are simply functions that are associated with particular objects. Technically, they are attributes attached to objects that happen to reference callable functions. In Python, expressions and built-in functions may work across a range of types, but methods are generally *specific to object types*—string methods, for example, work only on string objects. The method sets of some types intersect in Python 3.0 (e.g., many types have a `count` method), but they are still more type-specific than other tools.

In finer-grained detail, functions are packages of code, and method calls combine two operations at once (an attribute fetch and a call):

Attribute fetches

An expression of the form *object.attribute* means “fetch the value of *attribute* in *object*.”

Call expressions

An expression of the form *function(arguments)* means “invoke the code of *function*, passing zero or more comma-separated *argument* objects to it, and return *function*’s result value.”

Putting these two together allows us to call a method of an object. The method call expression *object.method(arguments)* is evaluated from left to right—Python will first fetch the *method* of the *object* and then call it, passing in the *arguments*. If the method computes a result, it will come back as the result of the entire method-call expression.

As you’ll see throughout this part of the book, most objects have callable methods, and all are accessed using this same method-call syntax. To call an object method, as you’ll see in the following sections, you have to go through an existing object.

[Table 7-3](#) summarizes the methods and call patterns for built-in string objects in Python 3.0; these change frequently, so be sure to check Python’s standard library manual for the most up-to-date list, or run a `help` call on any string interactively. Python 2.6’s string methods vary slightly; it includes a `decode`, for example, because of its different handling of Unicode data (something we’ll discuss in [Chapter 36](#)). In this table, *S* is a string object, and optional arguments are enclosed in square brackets. String methods in this table implement higher-level operations such as splitting and joining, case conversions, content tests, and substring searches and replacements.

Table 7-3. String method calls in Python 3.0

<code>S.capitalize()</code>	<code>S.ljust(width [, fill])</code>
<code>S.center(width [, fill])</code>	<code>S.lower()</code>
<code>S.count(sub [, start [, end]])</code>	<code>S.lstrip([chars])</code>
<code>S.encode([encoding [,errors]])</code>	<code>S.maketrans(x[, y[, z]])</code>
<code>S.endswith(suffix [, start [, end]])</code>	<code>S.partition(sep)</code>
<code>S.expandtabs([tabsize])</code>	<code>S.replace(old, new [, count])</code>
<code>S.find(sub [, start [, end]])</code>	<code>S.rfind(sub [,start [,end]])</code>
<code>S.format(fmtstr, *args, **kwargs)</code>	<code>S.rindex(sub [, start [, end]])</code>
<code>S.index(sub [, start [, end]])</code>	<code>S.rjust(width [, fill])</code>
<code>S.isalnum()</code>	<code>S.rpartition(sep)</code>
<code>S.isalpha()</code>	<code>S.rsplit([sep[, maxsplit]])</code>
<code>S.isdecimal()</code>	<code>S.rstrip([chars])</code>
<code>S.isdigit()</code>	<code>S.split([sep [,maxsplit]])</code>

<code>S.isidentifier()</code>	<code>S.splitlines([keepends])</code>
<code>S.islower()</code>	<code>S.startswith(prefix [, start [, end]])</code>
<code>S.isnumeric()</code>	<code>S.strip([chars])</code>
<code>S.isprintable()</code>	<code>S.swapcase()</code>
<code>S.isspace()</code>	<code>S.title()</code>
<code>S.istitle()</code>	<code>S.translate(map)</code>
<code>S.isupper()</code>	<code>S.upper()</code>
<code>S.join(iterable)</code>	<code>S.zfill(width)</code>

As you can see, there are quite a few string methods, and we don't have space to cover them all; see Python's library manual or reference texts for all the fine points. To help you get started, though, let's work through some code that demonstrates some of the most commonly used methods in action, and illustrates Python text-processing basics along the way.

String Method Examples: Changing Strings

As we've seen, because strings are immutable, they cannot be changed in-place directly. To make a new text value from an existing string, you construct a new string with operations such as slicing and concatenation. For example, to replace two characters in the middle of a string, you can use code like this:

```
>>> S = 'spammy'
>>> S = S[:3] + 'xx' + S[5:]
>>> S
'spaxxy'
```

But, if you're really just out to replace a substring, you can use the string `replace` method instead:

```
>>> S = 'spammy'
>>> S = S.replace('mm', 'xx')
>>> S
'spaxxy'
```

The `replace` method is more general than this code implies. It takes as arguments the original substring (of any length) and the string (of any length) to replace it with, and performs a global search and replace:

```
>>> 'aa$bb$cc$dd'.replace('$', 'SPAM')
'aaSPAMbbSPAMccSPAMdd'
```

In such a role, `replace` can be used as a tool to implement template replacements (e.g., in form letters). Notice that this time we simply printed the result, instead of assigning it to a name—you need to assign results to names only if you want to retain them for later use.

If you need to replace one fixed-size string that can occur at any offset, you can do a replacement again, or search for the substring with the string `find` method and then slice:

```
>>> S = 'xxxxSPAMxxxxSPAMxxxx'
>>> where = S.find('SPAM')           # Search for position
>>> where                                     # Occurs at offset 4
4
>>> S = S[:where] + 'EGGS' + S[(where+4):]
>>> S
'xxxxEGGSxxxxSPAMxxxx'
```

The `find` method returns the offset where the substring appears (by default, searching from the front), or `-1` if it is not found. As we saw earlier, it's a substring search operation just like the `in` expression, but `find` returns the position of a located substring.

Another option is to use `replace` with a third argument to limit it to a single substitution:

```
>>> S = 'xxxxSPAMxxxxSPAMxxxx'
>>> S.replace('SPAM', 'EGGS')       # Replace all
'xxxxEGGSxxxxEGGSxxxx'

>>> S.replace('SPAM', 'EGGS', 1)    # Replace one
'xxxxEGGSxxxxSPAMxxxx'
```

Notice that `replace` returns a new string object each time. Because strings are immutable, methods never really change the subject strings in-place, even if they are called “replace”!

The fact that concatenation operations and the `replace` method generate new string objects each time they are run is actually a potential downside of using them to change strings. If you have to apply many changes to a very large string, you might be able to improve your script's performance by converting the string to an object that does support in-place changes:

```
>>> S = 'spammy'
>>> L = list(S)
>>> L
['s', 'p', 'a', 'm', 'm', 'y']
```

The built-in `list` function (or an object construction call) builds a new list out of the items in any sequence—in this case, “exploding” the characters of a string into a list. Once the string is in this form, you can make multiple changes to it without generating a new copy for each change:

```
>>> L[3] = 'x'                       # Works for lists, not strings
>>> L[4] = 'x'
>>> L
['s', 'p', 'a', 'x', 'x', 'y']
```

If, after your changes, you need to convert back to a string (e.g., to write to a file), use the string `join` method to “implode” the list back into a string:

```
>>> S = ''.join(L)
>>> S
'spaxxy'
```

The `join` method may look a bit backward at first sight. Because it is a method of strings (not of lists), it is called through the desired delimiter. `join` puts the strings in a list (or other iterable) together, with the delimiter between list items; in this case, it uses an empty string delimiter to convert from a list back to a string. More generally, any string delimiter and iterable of strings will do:

```
>>> 'SPAM'.join(['eggs', 'sausage', 'ham', 'toast'])
'eggsSPAMsausageSPAMhamSPAMtoast'
```

In fact, joining substrings all at once this way often runs much faster than concatenating them individually. Be sure to also see the earlier note about the mutable `bytearray` string in Python 3.0 and 2.6, described fully in [Chapter 36](#); because it may be changed in place, it offers an alternative to this `list/join` combination for some kinds of text that must be changed often.

String Method Examples: Parsing Text

Another common role for string methods is as a simple form of text *parsing*—that is, analyzing structure and extracting substrings. To extract substrings at fixed offsets, we can employ slicing techniques:

```
>>> line = 'aaa bbb ccc'
>>> col1 = line[0:3]
>>> col3 = line[8:]
>>> col1
'aaa'
>>> col3
'ccc'
```

Here, the columns of data appear at fixed offsets and so may be sliced out of the original string. This technique passes for parsing, as long as the components of your data have fixed positions. If instead some sort of delimiter separates the data, you can pull out its components by splitting. This will work even if the data may show up at arbitrary positions within the string:

```
>>> line = 'aaa bbb ccc'
>>> cols = line.split()
>>> cols
['aaa', 'bbb', 'ccc']
```

The string `split` method chops up a string into a list of substrings, around a delimiter string. We didn't pass a delimiter in the prior example, so it defaults to whitespace—the string is split at groups of one or more spaces, tabs, and newlines, and we get back a list of the resulting substrings. In other applications, more tangible delimiters may separate the data. This example splits (and hence parses) the string at commas, a separator common in data returned by some database tools:

```
>>> line = 'bob,hacker,40'
>>> line.split(',')
['bob', 'hacker', '40']
```

Delimiters can be longer than a single character, too:

```
>>> line = "i'mSPAMaSPAMlumberjack"
>>> line.split("SPAM")
['i'm', 'a', 'lumberjack']
```

Although there are limits to the parsing potential of slicing and splitting, both run very fast and can handle basic text-extraction chores.

Other Common String Methods in Action

Other string methods have more focused roles—for example, to strip off whitespace at the end of a line of text, perform case conversions, test content, and test for a substring at the end or front:

```
>>> line = "The knights who say Ni!\n"
>>> line.rstrip()
'The knights who say Ni!'
>>> line.upper()
'THE KNIGHTS WHO SAY NI!\n'
>>> line.isalpha()
False
>>> line.endswith('Ni!\n')
True
>>> line.startswith('The')
True
```

Alternative techniques can also sometimes be used to achieve the same results as string methods—the `in` membership operator can be used to test for the presence of a substring, for instance, and length and slicing operations can be used to mimic `endswith`:

```
>>> line
'The knights who say Ni!\n'

>>> line.find('Ni') != -1      # Search via method call or expression
True
>>> 'Ni' in line
True

>>> sub = 'Ni!\n'
>>> line.endswith(sub)        # End test via method call or slice
True
>>> line[-len(sub):] == sub
True
```

See also the `format` string formatting method described later in this chapter; it provides more advanced substitution tools that combine many operations in a single step.

Again, because there are so many methods available for strings, we won't look at every one here. You'll see some additional string examples later in this book, but for more

details you can also turn to the Python library manual and other documentation sources, or simply experiment interactively on your own. You can also check the `help(S.method)` results for a *method* of any string object *S* for more hints.

Note that none of the string methods accepts *patterns*—for pattern-based text processing, you must use the Python `re` standard library module, an advanced tool that was introduced in [Chapter 4](#) but is mostly outside the scope of this text (one further example appears at the end of [Chapter 36](#)). Because of this limitation, though, string methods may sometimes run more quickly than the `re` module’s tools.

The Original string Module (Gone in 3.0)

The history of Python’s string methods is somewhat convoluted. For roughly the first decade of its existence, Python provided a standard library module called `string` that contained functions that largely mirrored the current set of string object methods. In response to user requests, in Python 2.0 these functions were made available as methods of string objects. Because so many people had written so much code that relied on the original `string` module, however, it was retained for backward compatibility.

Today, you should use *only string methods*, not the original `string` module. In fact, the original module-call forms of today’s string methods have been removed completely from Python in Release 3.0. However, because you may still see the module in use in older Python code, a brief look is in order here.

The upshot of this legacy is that in Python 2.6, there technically are still two ways to invoke advanced string operations: by calling object methods, or by calling `string` module functions and passing in the objects as arguments. For instance, given a variable *X* assigned to a string object, calling an object method:

```
X.method(arguments)
```

is usually equivalent to calling the same operation through the `string` module (provided that you have already imported the module):

```
string.method(X, arguments)
```

Here’s an example of the method scheme in action:

```
>>> S = 'a+b+c+'
>>> x = S.replace('+', 'spam')
>>> x
'aspambspamcspam'
```

To access the same operation through the `string` module in Python 2.6, you need to import the module (at least once in your process) and pass in the object:

```
>>> import string
>>> y = string.replace(S, '+', 'spam')
>>> y
'aspambspamcspam'
```

Because the module approach was the standard for so long, and because strings are such a central component of most programs, you might see both call patterns in Python 2.X code you come across.

Again, though, today you should always use method calls instead of the older module calls. There are good reasons for this, besides the fact that the module calls have gone away in Release 3.0. For one thing, the module call scheme requires you to import the `string` module (methods do not require imports). For another, the module makes calls a few characters longer to type (when you load the module with `import`, that is, not using `from`). And, finally, the module runs more slowly than methods (the module maps most calls back to the methods and so incurs an extra call along the way).

The original `string` module itself, without its string method equivalents, is retained in Python 3.0 because it contains additional tools, including predefined string constants and a template object system (a relatively obscure tool omitted here—see the Python library manual for details on template objects). Unless you really want to have to change your 2.6 code to use 3.0, though, you should consider the basic string operation calls in it to be just ghosts from the past.

String Formatting Expressions

Although you can get a lot done with the string methods and sequence operations we’ve already met, Python also provides a more advanced way to combine string processing tasks—*string formatting* allows us to perform multiple type-specific substitutions on a string in a single step. It’s never strictly required, but it can be convenient, especially when formatting text to be displayed to a program’s users. Due to the wealth of new ideas in the Python world, string formatting is available in two flavors in Python today:

String formatting expressions

The original technique, available since Python’s inception; this is based upon the C language’s “printf” model and is used in much existing code.

String formatting method calls

A newer technique added in Python 2.6 and 3.0; this is more unique to Python and largely overlaps with string formatting expression functionality.

Since the method call flavor is new, there is some chance that one or the other of these may become deprecated over time. The expressions are more likely to be deprecated in later Python releases, though this should depend on the future practice of real Python programmers. As they are largely just variations on a theme, though, either technique is valid to use today. Since string formatting expressions are the original in this department, let’s start with them.

Python defines the `%` binary operator to work on strings (you may recall that this is also the remainder of division, or modulus, operator for numbers). When applied to strings, the `%` operator provides a simple way to format values as strings according to a format

definition. In short, the % operator provides a compact way to code multiple string substitutions all at once, instead of building and concatenating parts individually.

To format strings:

1. On the left of the % operator, provide a format string containing one or more embedded conversion targets, each of which starts with a % (e.g., %d).
2. On the right of the % operator, provide the object (or objects, embedded in a tuple) that you want Python to insert into the format string on the left in place of the conversion target (or targets).

For instance, in the formatting example we saw earlier in this chapter, the integer 1 replaces the %d in the format string on the left, and the string 'dead' replaces the %s. The result is a new string that reflects these two substitutions:

```
>>> 'That is %d %s bird!' % (1, 'dead')           # Format expression
That is 1 dead bird!
```

Technically speaking, string formatting expressions are usually optional—you can generally do similar work with multiple concatenations and conversions. However, formatting allows us to combine many steps into a single operation. It's powerful enough to warrant a few more examples:

```
>>> exclamation = "Ni"
>>> "The knights who say %s!" % exclamation
'The knights who say Ni!'

>>> "%d %s %d you" % (1, 'spam', 4)
'1 spam 4 you'

>>> "%s -- %s -- %s" % (42, 3.14159, [1, 2, 3])
'42 -- 3.14159 -- [1, 2, 3]'
```

The first example here plugs the string "Ni" into the target on the left, replacing the %s marker. In the second example, three values are inserted into the target string. Note that when you're inserting more than one value, you need to group the values on the right in parentheses (i.e., put them in a tuple). The % formatting expression operator expects either a single item or a tuple of one or more items on its right side.

The third example again inserts three values—an integer, a floating-point object, and a list object—but notice that all of the targets on the left are %s, which stands for conversion to string. As every type of object can be converted to a string (the one used when printing), every object type works with the %s conversion code. Because of this, unless you will be doing some special formatting, %s is often the only code you need to remember for the formatting expression.

Again, keep in mind that formatting always makes a new string, rather than changing the string on the left; because strings are immutable, it must work this way. As before, assign the result to a variable name if you need to retain it.

Advanced String Formatting Expressions

For more advanced type-specific formatting, you can use any of the conversion type codes listed in [Table 7-4](#) in formatting expressions; they appear after the % character in substitution targets. C programmers will recognize most of these because Python string formatting supports all the usual C `printf` format codes (but returns the result, instead of displaying it, like `printf`). Some of the format codes in the table provide alternative ways to format the same type; for instance, %e, %f, and %g provide alternative ways to format floating-point numbers.

Table 7-4. String formatting type codes

Code	Meaning
s	String (or any object's <code>str(X)</code> string)
r	s, but uses <code>repr</code> , not <code>str</code>
c	Character
d	Decimal (integer)
i	Integer
u	Same as d (obsolete: no longer unsigned)
o	Octal integer
x	Hex integer
X	x, but prints uppercase
e	Floating-point exponent, lowercase
E	Same as e, but prints uppercase
f	Floating-point decimal
F	Floating-point decimal
g	Floating-point e or f
G	Floating-point E or F
%	Literal %

In fact, conversion targets in the format string on the expression's left side support a variety of conversion operations with a fairly sophisticated syntax all their own. The general structure of conversion targets looks like this:

```
%(name)[flags][width][.precision]typecode
```

The character type codes in [Table 7-4](#) show up at the end of the target string. Between the % and the character code, you can do any of the following: provide a dictionary key; list flags that specify things like left justification (-), numeric sign (+), and zero fills (0); give a total minimum field width and the number of digits after a decimal point; and more. Both *width* and *precision* can also be coded as a * to specify that they should take their values from the next item in the input values.

Formatting target syntax is documented in full in the Python standard manuals, but to demonstrate common usage, let's look at a few examples. This one formats integers by default, and then in a six-character field with left justification and zero padding:

```
>>> x = 1234
>>> res = "integers: ...%d...%-6d...%06d" % (x, x, x)
>>> res
'integers: ...1234...1234  ...001234'
```

The %e, %f, and %g formats display floating-point numbers in different ways, as the following interaction demonstrates (%E is the same as %e but the exponent is uppercase):

```
>>> x = 1.23456789
>>> x
1.2345678899999999

>>> '%e | %f | %g' % (x, x, x)
'1.234568e+00 | 1.234568 | 1.23457'

>>> '%E' % x
'1.234568E+00'
```

For floating-point numbers, you can achieve a variety of additional formatting effects by specifying left justification, zero padding, numeric signs, field width, and digits after the decimal point. For simpler tasks, you might get by with simply converting to strings with a format expression or the `str` built-in function shown earlier:

```
>>> '%-6.2f | %05.2f | %+06.1f' % (x, x, x)
'1.23   | 01.23 | +001.2'

>>> "%s" % x, str(x)
('1.23456789', '1.23456789')
```

When sizes are not known until runtime, you can have the width and precision computed by specifying them with a * in the format string to force their values to be taken from the next item in the inputs to the right of the % operator—the 4 in the tuple here gives precision:

```
>>> '%f, %.2f, %.*f' % (1/3.0, 1/3.0, 4, 1/3.0)
'0.333333, 0.33, 0.3333'
```

If you're interested in this feature, experiment with some of these examples and operations on your own for more information.

Dictionary-Based String Formatting Expressions

String formatting also allows conversion targets on the left to refer to the keys in a dictionary on the right and fetch the corresponding values. I haven't told you much about dictionaries yet, so here's an example that demonstrates the basics:

```
>>> "%(n)d %(x)s" % {"n":1, "x":"spam"}
'1 spam'
```

Here, the (n) and (x) in the format string refer to keys in the dictionary literal on the right and fetch their associated values. Programs that generate text such as HTML or XML often use this technique—you can build up a dictionary of values and substitute them all at once with a single formatting expression that uses key-based references:

```
>>> reply = ""                                     # Template with substitution targets
Greetings...
Hello %(name)s!
Your age squared is %(age)s
"""

>>> values = {'name': 'Bob', 'age': 40}             # Build up values to substitute
>>> print(reply % values)                           # Perform substitutions

Greetings...
Hello Bob!
Your age squared is 40
```

This trick is also used in conjunction with the `vars` built-in function, which returns a dictionary containing all the variables that exist in the place it is called:

```
>>> food = 'spam'
>>> age = 40
>>> vars()
{'food': 'spam', 'age': 40, ...many more... }
```

When used on the right of a format operation, this allows the format string to refer to variables by name (i.e., by dictionary key):

```
>>> "%(age)d %(food)s" % vars()
'40 spam'
```

We'll study dictionaries in more depth in [Chapter 8](#). See also [Chapter 5](#) for examples that convert to hexadecimal and octal number strings with the `%x` and `%o` formatting target codes.

String Formatting Method Calls

As mentioned earlier, Python 2.6 and 3.0 introduced a new way to format strings that is seen by some as a bit more Python-specific. Unlike formatting expressions, formatting method calls are not closely based upon the C language's "printf" model, and they are more verbose and explicit in intent. On the other hand, the new technique still relies on some "printf" concepts, such as type codes and formatting specifications. Moreover, it largely overlaps with (and sometimes requires a bit more code than) formatting expressions, and it can be just as complex in advanced roles. Because of this, there is no best-use recommendation between expressions and method calls today, so most programmers would be well served by a cursory understanding of both schemes.

The Basics

In short, the new string object's `format` method in 2.6 and 3.0 (and later) uses the subject string as a template and takes any number of arguments that represent values to be substituted according to the template. Within the subject string, curly braces designate substitution targets and arguments to be inserted either by position (e.g., `{1}`) or keyword (e.g., `{food}`). As we'll learn when we study argument passing in depth in [Chapter 18](#), arguments to functions and methods may be passed by position or keyword name, and Python's ability to collect arbitrarily many positional and keyword arguments allows for such general method call patterns. In Python 2.6 and 3.0, for example:

```
>>> template = '{0}, {1} and {2}'                                     # By position
>>> template.format('spam', 'ham', 'eggs')
'spam, ham and eggs'

>>> template = '{motto}, {pork} and {food}'                           # By keyword
>>> template.format(motto='spam', pork='ham', food='eggs')
'spam, ham and eggs'

>>> template = '{motto}, {0} and {food}'                               # By both
>>> template.format('ham', motto='spam', food='eggs')
'spam, ham and eggs'
```

Naturally, the string can also be a literal that creates a temporary string, and arbitrary object types can be substituted:

```
>>> '{motto}, {0} and {food}'.format(42, motto=3.14, food=[1, 2])
'3.14, 42 and [1, 2]'
```

Just as with the `%` expression and other string methods, `format` creates and returns a new string object, which can be printed immediately or saved for further work (recall that strings are immutable, so `format` really *must* make a new object). String formatting is not just for display:

```
>>> X = '{motto}, {0} and {food}'.format(42, motto=3.14, food=[1, 2])
>>> X
'3.14, 42 and [1, 2]'
```

```
>>> X.split(' and ')
['3.14, 42', '[1, 2]']
```

```
>>> Y = X.replace('and', 'but under no circumstances')
>>> Y
'3.14, 42 but under no circumstances [1, 2]'
```

Adding Keys, Attributes, and Offsets

Like `%` formatting expressions, `format` calls can become more complex to support more advanced usage. For instance, `format` strings can name object attributes and dictionary keys—as in normal Python syntax, square brackets name dictionary keys and dots denote object attributes of an item referenced by position or keyword. The first of the

following examples indexes a dictionary on the key “spam” and then fetches the attribute “platform” from the already imported `sys` module object. The second does the same, but names the objects by keyword instead of position:

```
>>> import sys

>>> 'My {1[spam]} runs {0.platform}'.format(sys, {'spam': 'laptop'})
'My laptop runs win32'

>>> 'My {config[spam]} runs {sys.platform}'.format(sys=sys,
                                                    config={'spam': 'laptop'})
'My laptop runs win32'
```

Square brackets in format strings can name list (and other sequence) offsets to perform indexing, too, but only single positive offsets work syntactically within format strings, so this feature is not as general as you might think. As with `%` expressions, to name negative offsets or slices, or to use arbitrary expression results in general, you must run expressions outside the format string itself:

```
>>> somelist = list('SPAM')
>>> somelist
['S', 'P', 'A', 'M']

>>> 'first={0[0]}, third={0[2]}'.format(somelist)
'first=S, third=A'

>>> 'first={0}, last={1}'.format(somelist[0], somelist[-1])  # [-1] fails in fmt
'first=S, last=M'

>>> parts = somelist[0], somelist[-1], somelist[1:3]          # [1:3] fails in fmt
>>> 'first={0}, last={1}, middle={2}'.format(*parts)
'first=S, last=M, middle=['P', 'A']"
```

Adding Specific Formatting

Another similarity with `%` expressions is that more specific layouts can be achieved by adding extra syntax in the format string. For the formatting method, we use a colon after the substitution target’s identification, followed by a format specifier that can name the field size, justification, and a specific type code. Here’s the formal structure of what can appear as a substitution target in a format string:

```
{fieldname!conversionflag:formatspec}
```

In this substitution target syntax:

- *fieldname* is a number or keyword naming an argument, followed by optional “name” attribute or “[index]” component references.
- *conversionflag* can be `r`, `s`, or `a` to call `repr`, `str`, or `ascii` built-in functions on the value, respectively.

- *formatspec* specifies how the value should be presented, including details such as field width, alignment, padding, decimal precision, and so on, and ends with an optional data type code.

The *formatspec* component after the colon character is formally described as follows (brackets denote optional components and are not coded literally):

```
[[fill]align][sign][#][0][width][.precision][typecode]
```

align may be <, >, =, or ^, for left alignment, right alignment, padding after a sign character, or centered alignment, respectively. The *formatspec* also contains nested {} format strings with field names only, to take values from the arguments list dynamically (much like the * in formatting expressions).

See Python’s library manual for more on substitution syntax and a list of the available type codes—they almost completely overlap with those used in % expressions and listed previously in Table 7-4, but the format method also allows a “b” type code used to display integers in binary format (it’s equivalent to using the `bin` built-in call), allows a “%” type code to display percentages, and uses only “d” for base-10 integers (not “i” or “u”).

As an example, in the following {0:10} means the first positional argument in a field 10 characters wide, {1:<10} means the second positional argument left-justified in a 10-character-wide field, and {0.platform:>10} means the `platform` attribute of the first argument right-justified in a 10-character-wide field:

```
>>> '{0:10} = {1:10}'.format('spam', 123.4567)
'spam      =    123.457'

>>> '{0:>10} = {1:<10}'.format('spam', 123.4567)
'      spam = 123.457 '

>>> '{0.platform:>10} = {1[item]:<10}'.format(sys, dict(item='laptop'))
'      win32 = laptop'
```

Floating-point numbers support the same type codes and formatting specificity in formatting method calls as in % expressions. For instance, in the following {2:g} means the third argument formatted by default according to the “g” floating-point representation, {1:.2f} designates the “f” floating-point format with just 2 decimal digits, and {2:06.2f} adds a field with a width of 6 characters and zero padding on the left:

```
>>> '{0:e}, {1:.3e}, {2:g}'.format(3.14159, 3.14159, 3.14159)
'3.141590e+00, 3.142e+00, 3.14159'

>>> '{0:f}, {1:.2f}, {2:06.2f}'.format(3.14159, 3.14159, 3.14159)
'3.141590, 3.14, 003.14'
```

Hex, octal, and binary formats are supported by the `format` method as well. In fact, string formatting is an alternative to some of the built-in functions that format integers to a given base:

```

>>> '{0:X}, {1:o}, {2:b}'.format(255, 255, 255)    # Hex, octal, binary
'FF, 377, 11111111'

>>> bin(255), int('11111111', 2), 0b11111111      # Other to/from binary
('0b11111111', 255, 255)

>>> hex(255), int('FF', 16), 0xFF                 # Other to/from hex
('0xff', 255, 255)

>>> oct(255), int('377', 8), 0o377, 0377          # Other to/from octal
('0377', 255, 255, 255)                          # 0377 works in 2.6, not 3.0!

```

Formatting parameters can either be hardcoded in format strings or taken from the arguments list dynamically by nested format syntax, much like the star syntax in formatting expressions:

```

>>> '{0:.2f}'.format(1 / 3.0)                      # Parameters hardcoded
'0.33'
>>> '%.2f' % (1 / 3.0)
'0.33'

>>> '{0:.{1}f}'.format(1 / 3.0, 4)                 # Take value from arguments
'0.3333'
>>> '%.*f' % (4, 1 / 3.0)                          # Ditto for expression
'0.3333'

```

Finally, Python 2.6 and 3.0 also provide a new built-in `format` function, which can be used to format a single item. It's a more concise alternative to the string `format` method, and is roughly similar to formatting a single item with the `%` formatting expression:

```

>>> '{0:.2f}'.format(1.2345)                       # String method
'1.23'
>>> format(1.2345, '.2f')                          # Built-in function
'1.23'
>>> '%.2f' % 1.2345                                 # Expression
'1.23'

```

Technically, the `format` built-in runs the subject object's `__format__` method, which the `str.format` method does internally for each formatted item. It's still more verbose than the original `%` expression's equivalent, though—which leads us to the next section.

Comparison to the % Formatting Expression

If you study the prior sections closely, you'll probably notice that at least for positional references and dictionary keys, the string `format` method looks very much like the `%` formatting expression, especially in advanced use with type codes and extra formatting syntax. In fact, in common use cases formatting expressions may be easier to code than formatting method calls, especially when using the generic `%s` print-string substitution target:

```

print('%s=%s' % ('spam', 42))                      # 2.X+ format expression

print('{0}={1}'.format('spam', 42))                # 3.0 (and 2.6) format method

```

As we'll see in a moment, though, more complex formatting tends to be a draw in terms of complexity (difficult tasks are generally difficult, regardless of approach), and some see the formatting method as largely redundant.

On the other hand, the formatting method also offers a few potential advantages. For example, the original % expression can't handle keywords, attribute references, and binary type codes, although dictionary key references in % format strings can often achieve similar goals. To see how the two techniques overlap, compare the following % expressions to the equivalent `format` method calls shown earlier:

```
# The basics: with % instead of format()

>>> template = '%s, %s, %s'
>>> template % ('spam', 'ham', 'eggs')           # By position
'spam, ham, eggs'

>>> template = '%(motto)s, %(pork)s and %(food)s'
>>> template % dict(motto='spam', pork='ham', food='eggs')  # By key
'spam, ham and eggs'

>>> '%s, %s and %s' % (3.14, 42, [1, 2])          # Arbitrary types
'3.14, 42 and [1, 2]'

# Adding keys, attributes, and offsets

>>> 'My %(spam)s runs %(platform)s' % {'spam': 'laptop', 'platform': sys.platform}
'My laptop runs win32'

>>> 'My %(spam)s runs %(platform)s' % dict(spam='laptop', platform=sys.platform)
'My laptop runs win32'

>>> somelist = list('SPAM')
>>> parts = somelist[0], somelist[-1], somelist[1:3]
>>> 'first=%s, last=%s, middle=%s' % parts
'first=S, last=M, middle=[P, A]'
```

When more complex formatting is applied the two techniques approach parity in terms of complexity, although if you compare the following with the `format` method call equivalents listed earlier you'll again find that the % expressions tend to be a bit simpler and more concise:

```
# Adding specific formatting

>>> '%-10s = %10s' % ('spam', 123.4567)
'spam          = 123.4567'

>>> '%10s = %-10s' % ('spam', 123.4567)
'      spam = 123.4567 '

>>> '%(plat)10s = %(item)-10s' % dict(plat=sys.platform, item='laptop')
'      win32 = laptop '
```

Floating-point numbers

```
>>> '%e, %.3e, %g' % (3.14159, 3.14159, 3.14159)
'3.141590e+00, 3.142e+00, 3.14159'

>>> '%f, %.2f, %06.2f' % (3.14159, 3.14159, 3.14159)
'3.141590, 3.14, 003.14'
```

Hex and octal, but not binary

```
>>> '%x, %o' % (255, 255)
'ff, 377'
```

The `format` method has a handful of advanced features that the `%` expression does not, but even more involved formatting still seems to be essentially a draw in terms of complexity. For instance, the following shows the same result generated with both techniques, with field sizes and justifications and various argument reference methods:

Hardcoded references in both

```
>>> import sys

>>> 'My {1[spam]:<8} runs {0.platform:>8}'.format(sys, {'spam': 'laptop'})
'My laptop runs win32'

>>> 'My %(spam)-8s runs %(plat)8s' % dict(spam='laptop', plat=sys.platform)
'My laptop runs win32'
```

In practice, programs are less likely to hardcode references like this than to execute code that builds up a set of substitution data ahead of time (to collect data to substitute into an HTML template all at once, for instance). When we account for common practice in examples like this, the comparison between the `format` method and the `%` expression is even more direct (as we'll see in [Chapter 18](#), the `**data` in the method call here is special syntax that unpacks a dictionary of keys and values into individual “name=value” keyword arguments so they can be referenced by name in the format string):

Building data ahead of time in both

```
>>> data = dict(platform=sys.platform, spam='laptop')

>>> 'My {spam:<8} runs {platform:>8}'.format(**data)
'My laptop runs win32'

>>> 'My %(spam)-8s runs %(platform)8s' % data
'My laptop runs win32'
```

As usual, the Python community will have to decide whether `%` expressions, `format` method calls, or a toolset with both techniques proves better over time. Experiment with these techniques on your own to get a feel for what they offer, and be sure to see the Python 2.6 and 3.0 library manuals for more details.